

FeatureLeakageLens

FeatureLeakageLens / Interview Defense Document

Sidharth Kriplani

Document v1.0 · Sourced exclusively from repo files

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces verified artifact or DB row
[SIMULATED]	Deliberately synthetic scenario; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists but verification relies on internal consistency only
[FUTURE]	Not built — explicitly absent from the repo

FeatureLeakageLens Interview Defense Document v2.0

Complete Edition — Pre-Training Feature Leakage Auditing

SECTION 1 — PROJECT IDENTITY

1.1 One-Sentence Summary

FeatureLeakageLens is a pre-training tabular ML auditor that runs six structured checks — post-outcome name heuristics, ID/proxy scans, target correlation, categorical proxy detection, split distribution shift, and temporal availability — and produces a per-feature PASS/WARN/FAIL report before a single model is trained.

1.2 The Problem This Solves

Feature leakage is the single most dangerous silent failure in supervised ML. A model trained on a post-outcome column doesn't look miscalibrated — it looks exceptional. AUC near 1.0, precision through the roof, validation loss flatlining at epoch 2. Nothing in the training curves signals the problem. The failure surfaces in production when the feature isn't available because it was generated after the event you were predicting. By then, the model has shipped, the business has made decisions on it, and months of development are sunk.

FeatureLeakageLens makes the review step explicit, systematic, and documentable. Six checks run before fitting any model. The output is a structured report attachable to a model card or pull request review.

1.3 Truth Boundary

FeatureLeakageLens flags suspicious patterns. It does not prove leakage. Every finding requires domain expert confirmation. Only the temporal availability check (`feature_ts > outcome_ts` per row) produces a FAIL — this is the one check with no ambiguity. All other checks produce WARN.

SECTION 2 — THE SIX CHECKS

2.1 Check Architecture

Tier	Check	Method	Max Status		
Name & Structure	Post-outcome name heuristic	Keyword scan on column names	WARN		
Name & Structure	ID / proxy scan	<code>n_unique / n_rows ≥ id_threshold</code>	WARN		
Statistical	Target correlation scan	Pearson χ^2	$r \geq \text{high_corr_thresh}$	WARN	
Statistical	Categorical proxy scan	<code>max_rate - min_rate ≥ cat_proxy_threshold</code>	WARN		
Statistical	Split distribution scan	Normalised mean diff (numeric) + TVD (categorical)	WARN or INSUFFICIENT_INPUT		
Temporal	Temporal availability	<code>feature_ts > outcome_ts</code> per row	FAIL		

2.2 Post-Outcome Name Heuristic [IMPLEMENTED]

The first check scans column names for keyword patterns associated with post-outcome features. The keyword list includes terms like `outcome`, `result`, `final`, `post`, `after`, `completed`, `resolved`, `churned`, `defaulted`, `paid`, `repaid`, `closed`. Any feature column whose name contains one of these terms is flagged WARN with the matching keyword as evidence.

This check is zero-compute — it runs on column names alone before a single row of data is read. It is intentionally low-precision (high WARN rate) because the cost of a false negative (missing leakage) vastly exceeds the cost of a false positive (a human reviews a column name).

2.3 ID / Proxy Scan [IMPLEMENTED]

High-cardinality columns where $n_unique / n_rows \geq id_threshold$ (default: 0.95) are flagged as potential ID or proxy columns. IDs are dangerous leakage sources in two ways: (1) direct leakage — if an ID was assigned post-outcome, it encodes the label, (2) indirect leakage — if ID ranges correlate with time periods, they may encode temporal information that leaks the target distribution.

The check uses a configurable threshold so teams with UUID-format IDs (always cardinality 1.0) can set the threshold higher, while teams with customer IDs (cardinality ~0.5) can set it lower.

2.4 Target Correlation Scan [IMPLEMENTED]

For every numeric feature, Pearson $|r|$ is computed against the target column. Features with $|r| \geq high_corr_threshold$ (default: 0.7) are flagged WARN. A correlation of 0.7+ in a real production dataset is extremely rare for legitimate features — it almost always indicates the feature encodes the label directly or through a post-outcome proxy.

The check also handles binary targets correctly: for a binary $\{0, 1\}$ target, Pearson correlation is equivalent to the point-biserial correlation, which correctly measures discrimination power.

2.5 Categorical Proxy Scan [IMPLEMENTED]

For every categorical feature (non-numeric column with low cardinality), the check computes the target positive rate within each category value. If $max_rate - min_rate \geq cat_proxy_threshold$

(default: 0.5), the feature is flagged WARN. A 50-point gap in default rate across category values (e.g., category A defaults at 2%, category B defaults at 65%) is a strong signal that the categorical feature encodes outcome information.

This check does not flag features with genuine predictive power that fall below the threshold — it specifically targets near-perfect target rate separation that exceeds what legitimate features produce.

2.6 Split Distribution Scan [IMPLEMENTED]

This check computes the distribution shift between train and test splits for every feature. For numeric features, the check uses normalised mean difference: $|\text{mean_train} - \text{mean_test}| / \text{pooled_std}$. For categorical features, it uses Total Variation Distance (TVD): $0.5 \times \sum |P_{\text{train}}(v) - P_{\text{test}}(v)|$.

Features with shift above threshold are flagged WARN: split-correlated features can be artifacts of temporal train/test splits where post-outcome signals appear more in the later (test) period.

If no `split_col` is provided, this check returns `INSUFFICIENT_INPUT` rather than `PASS` — the distinction matters for documentation purposes.

2.7 Temporal Availability [IMPLEMENTED — Only FAIL check]

The most critical check. For each row, the check verifies that every feature timestamp `feature_ts` is $\leq$ `outcome_ts` (the event timestamp). If any row shows `feature_ts > outcome_ts`, the feature is flagged **FAIL**. This is the only unambiguous leakage finding: a feature computed after the outcome occurred cannot have been available at prediction time.

The FAIL status (not WARN) reflects the severity: temporal availability violations are not "suspicious patterns" — they are structural impossibilities in a properly designed prediction pipeline.

SECTION 3 — API AND USAGE

3.1 Core API

```
from featureleakagelens import LeakageAuditor, LeakageAuditConfig config = LeakageAuditConfig(
target_col="default", split_col="split", # "train" / "test" column outcome_time_col="loan_date",
feature_time_cols=["score_date", "employment_check_date"], high_corr_threshold=0.7,
cat_proxy_threshold=0.5, id_threshold=0.95, ) auditor = LeakageAuditor(df=df, config=config) report
= auditor.audit() print(report.overall_status) # PASS / WARN / FAIL print(report.to_markdown())
report.to_json("leakage_audit.json") report.to_html("leakage_audit.html")
```

3.2 Output Structure

Every report contains: `overall_status` (FAIL > WARN > INSUFFICIENT_INPUT > PASS), `findings` (list of per-feature findings with check name, status, evidence), `summary_counts` (counts by status), and `truth_boundary` (explicit statement that this tool flags patterns, not proven leakage).

The truth boundary text is mandatory and cannot be suppressed from any output format. This is a deliberate design decision: reports that omit the interpretation caveat get used as proof of no leakage, which is incorrect.

SECTION 4 — VERSION HISTORY

4.1 v0.4.0 (Current) — Temporal Boundary Scan + Weighted Risk Score [IMPLEMENTED]

Added temporal boundary scan: a new check that analyzes whether numeric features show discontinuous distribution shifts at the outcome boundary time. Features that are perfectly uniform before the outcome timestamp but show a sharp distribution shift after are flagged as potential temporal leakage even without per-row timestamps (using population-level temporal analysis).

Added weighted risk score: each finding is assigned a risk weight based on check type and severity. The report now includes a per-feature `risk_score` (0–10) in addition to the status flag, allowing teams to prioritize the most dangerous findings.

4.2 v0.3.0 — Categorical Proxy Enhancement

Expanded the categorical proxy scan to handle high-cardinality categoricals (>50 unique values) using sampling-based target rate estimation rather than full enumeration. This prevents the check from timing out on columns like zip codes or product IDs.

4.3 v0.2.0 — Split Distribution + HTML Output

Added split distribution scan (the 5th check) and HTML report format. The HTML output includes a color-coded findings table with FAIL in red, WARN in amber, and PASS in green, matching the audit report color conventions.

4.4 v0.1.0 — Initial Release

First four checks: name heuristic, ID scan, target correlation, categorical proxy. JSON and Markdown output formats. Basic CLI interface.

SECTION 5 — DESIGN DECISIONS

5.1 Why WARN for Most Checks, FAIL Only for Temporal

The decision to reserve FAIL for only temporal availability reflects a core philosophy: most leakage signals are probabilistic, not deterministic. A feature named "resolved_date" might be a post-outcome feature, or it might be a pre-outcome resolution step in the business process. A feature with 0.8 correlation with the target might be genuine (an incredibly predictive feature) or leaked. Only temporal availability — `feature_ts > outcome_ts` — has no legitimate explanation.

The WARN / FAIL distinction matters for adoption: a tool that flags everything as FAIL creates alarm fatigue and gets ignored. A tool that fires FAIL only on genuine structural violations gets taken seriously.

5.2 Why No Automatic Feature Removal

FeatureLeakageLens produces a report; it does not modify the DataFrame. This is intentional. Automated feature removal based on leakage heuristics would silently delete legitimate features (false positives). Instead, every finding requires a human decision: review the finding, determine whether it represents actual leakage, and remove the feature manually if confirmed. This keeps the human in the loop for irreversible data decisions.

5.3 Zero-Dependency Core

The core audit logic (`LeakageAuditor`) has no external dependencies beyond NumPy and pandas. This design makes the library installable in any ML environment without dependency conflicts. Optional HTML output uses only Python's `html` module from the standard library.

SECTION 6 — Q&A

Q: What's the most common leakage pattern you'd catch that others miss?

The categorical proxy scan. Teams routinely check numeric correlations but don't systematically check categorical target rates. A `loan_status_category` column with values like "active", "pending_review", and "written_off" will have a target rate of 0% for active, 15% for pending review, and 100% for written_off. The written_off category is a perfect target proxy — completely invisible to Pearson correlation (it's not numeric) and invisible to name heuristics (the name sounds legitimate). The categorical proxy scan catches this.

Q: How does FeatureLeakageLens handle high-cardinality categoricals like zip codes?

In v0.3.0+, high-cardinality categoricals (>50 unique values) are handled with sampling: rather than computing target rate for all values, the check samples 50 values and estimates the `max_rate - min_rate` gap from the sample. This trades exactness for speed, which is acceptable because leakage at the extreme end of a 50-value sample is representative of the population-level gap.

Q: What happens if you call it without a split_col?

The split distribution scan returns `INSUFFICIENT_INPUT` instead of `PASS`. This distinction is documented in the report and is intentional: "no split data provided" is not the same as "no distribution shift found." A downstream consumer reading the report should know that this check was not evaluable, not that it passed.

Q: How does the tool fit into a CI pipeline?

The CLI exit code follows the overall status: `PASS` = 0, `WARN` = 1, `FAIL` = 2, `INSUFFICIENT_INPUT` = 3. This makes it directly usable as a pre-training gate in CI: a `FAIL` status blocks model training; a `WARN` status requires a documented override comment in the PR.

Q: What leakage patterns does it NOT catch?

FeatureLeakageLens does not catch: (1) leakage through complex feature interactions (two individually legitimate features whose product encodes the target), (2) label-in-text leakage in NLP features (a description field mentioning "defaulted"), (3) leakage from auxiliary tables joined after the split, or (4) future-peeking in rolling aggregations (e.g., a 7-day rolling average computed on the full dataset before the split). These require more specialized tools or human review of the feature engineering code.

SECTION 7 — NUMBERS AT A GLANCE

Metric	Value
Checks	6
Checks that can FAIL	1 (temporal availability only)
Checks that WARN	5
Current version	v0.4.0
Test cases	13
Output formats	JSON, Markdown, HTML

Metric	Value
Core dependencies	NumPy, pandas only

FeatureLeakageLens Interview Defense Document v2.0 — Complete Edition

Six Checks · WARN/FAIL Distinction · Temporal Availability · Pre-Training Gate